

TileLang: High-Performance AI Compilation via Tile Programming

Zhi Yang

Peking University



TileLang Background

Writing Next-generation Operators Is Getting Harder

AI algorithms are increasingly architecture-aware: designed from the ground up with hardware characteristics in mind.

Hardware architectures are becoming complex, making programming more challenging.



MLA

DeepEP

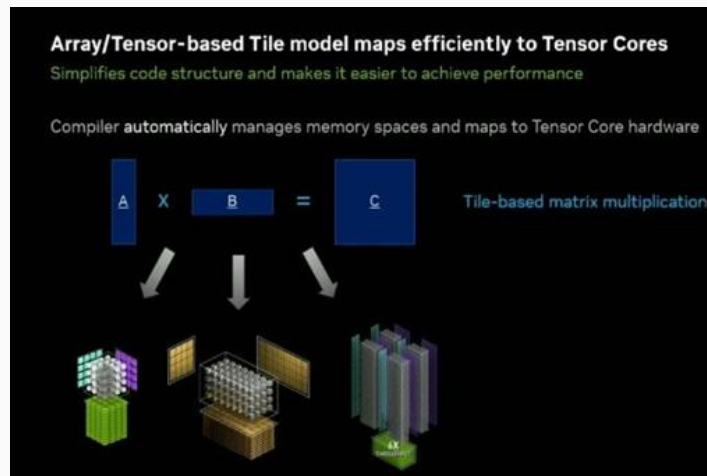
SparseAttention

Linear Attention

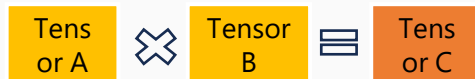
GDN

Diffusion

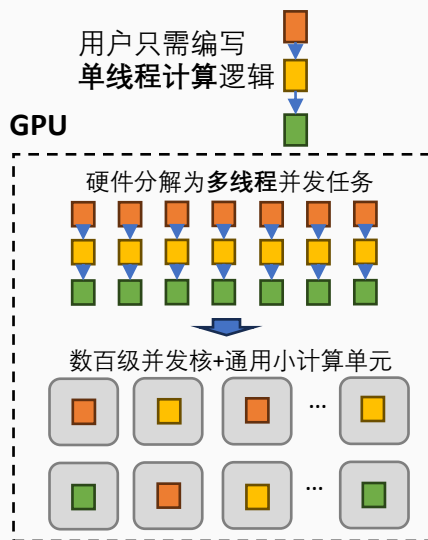
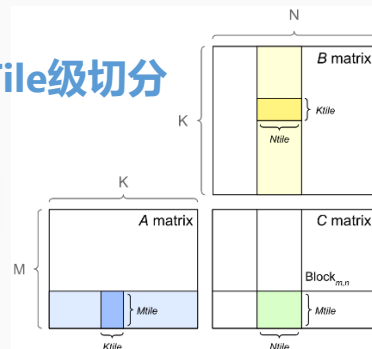
DiT/Vit



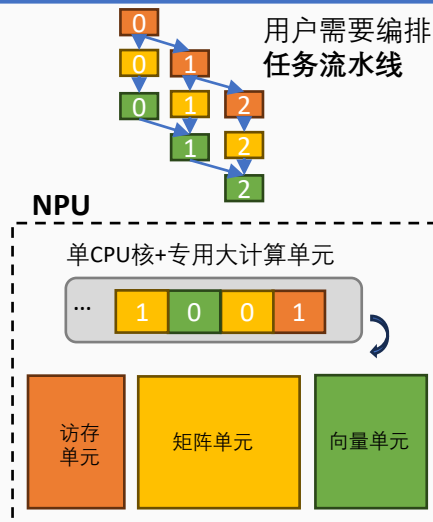
算子逻辑



Tile级切分



GPU	NPU
编程简单 开发者可快速实现新算子	编程复杂 新算子调优需要较长时间
并行效率高 硬件自动完成并发任务调度	并行效率低 并发任务依赖程序实现软件流水线
通用性强 对新模型或通用算法支持好	专用任务效率高 针对特定算子计算密度更高
硬件实现复杂 需自研GPU核及系统架构	硬件实现简单 可扩展现有开源CPU架构



Trade-offs in Operator Programming

Future accelerator programming must offer a structured and highly controllable approach.

Striking a balance between ease of development and low-level control over critical performance paths.



TileLang Programming Language

TileLang is an **open-source, community-driven tile programming language**.

Join us on [GitHub: tile-ai/tilelang](https://github.com/tile-ai/tilelang).



TileLang
TILE LANGUAGE

Tile Language

Tile Language (tile-lang) is a concise domain-specific language designed to streamline the development of high-performance GPU/CPU kernels (e.g., GEMM, Dequant GEMM, FlashAttention, LinearAttention). By employing a Pythonic syntax with an underlying compiler infrastructure on top of [TVM](#), tile-lang allows developers to focus on productivity without sacrificing the low-level optimizations necessary for state-of-the-art performance.

TileLang Architecture:

- Global Memory
- Shared Memory
- Register Files

The diagram illustrates the memory hierarchy: Register Files (top), Shared Memory (middle), and Global Memory (bottom). A grid of colored squares represents the data flow and memory access patterns between these components.

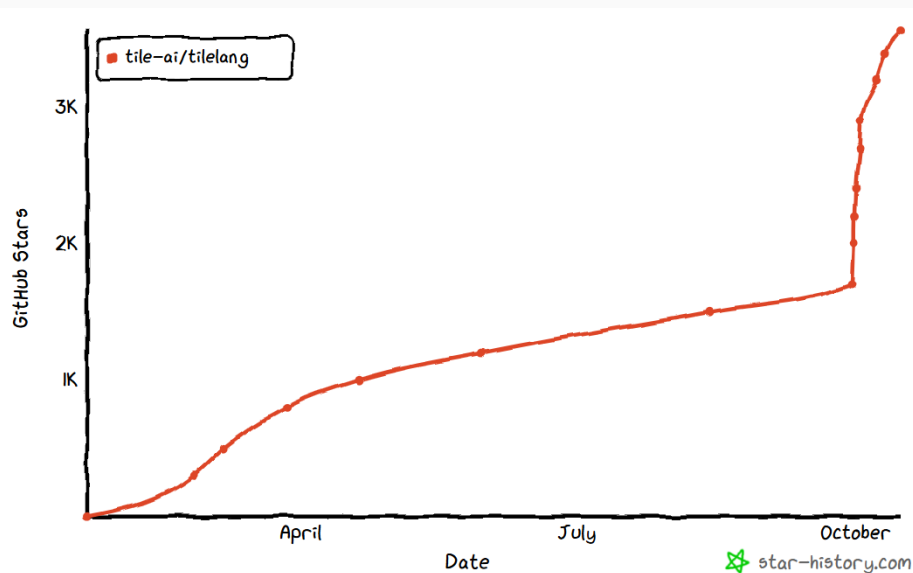
```
import tilelang.language as T
import numpy as np

# Kernel Control Initialization
def main():
    # Buffer Allocation
    A = T.tensor(...)
    B = T.tensor(...)
    C = T.tensor(...)
    D = T.tensor(...)

    # Main Loop with Pipeline Annotation
    @T.primop("gemm")
    def gemm(A, B, C):
        # Copy Data from Global to Shared Memory
        A_shared = T.alloc_shared(A)
        B_shared = T.alloc_shared(B)

        # Compute
        C_shared = T.alloc_shared(C)

        # Write Back to Global Memory
        C_shared.copy_to(C)
```





TileLang Design

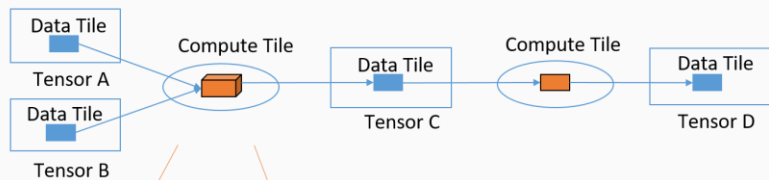
TileLang=Tile Virtual Machine + Control APIs

Tile-VM = Tile Task Model + Tile Hardware Abstraction

Hierarchical tile tasks across memory layers—small tiles to registers, medium tiles to SMEM, and large tiles to GMEM.

Tile-VM executes and transfers at tile-task granularity — aligned control of memory locality, compute scheduling, and data movement.

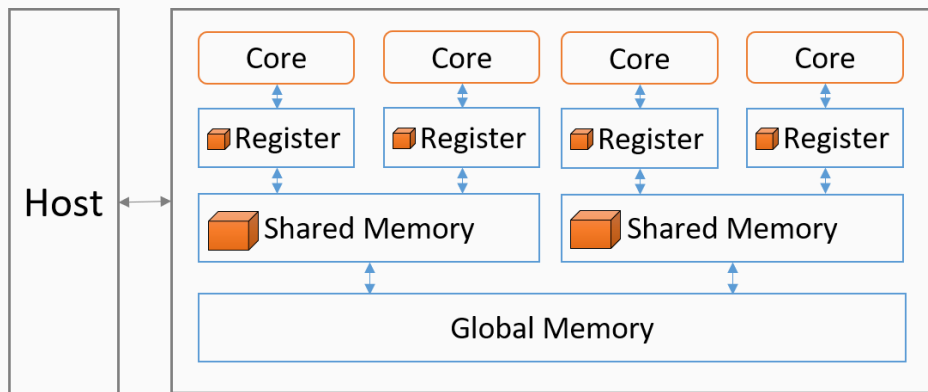
Hierarchical Tile Task



Hierarchical
parallelism

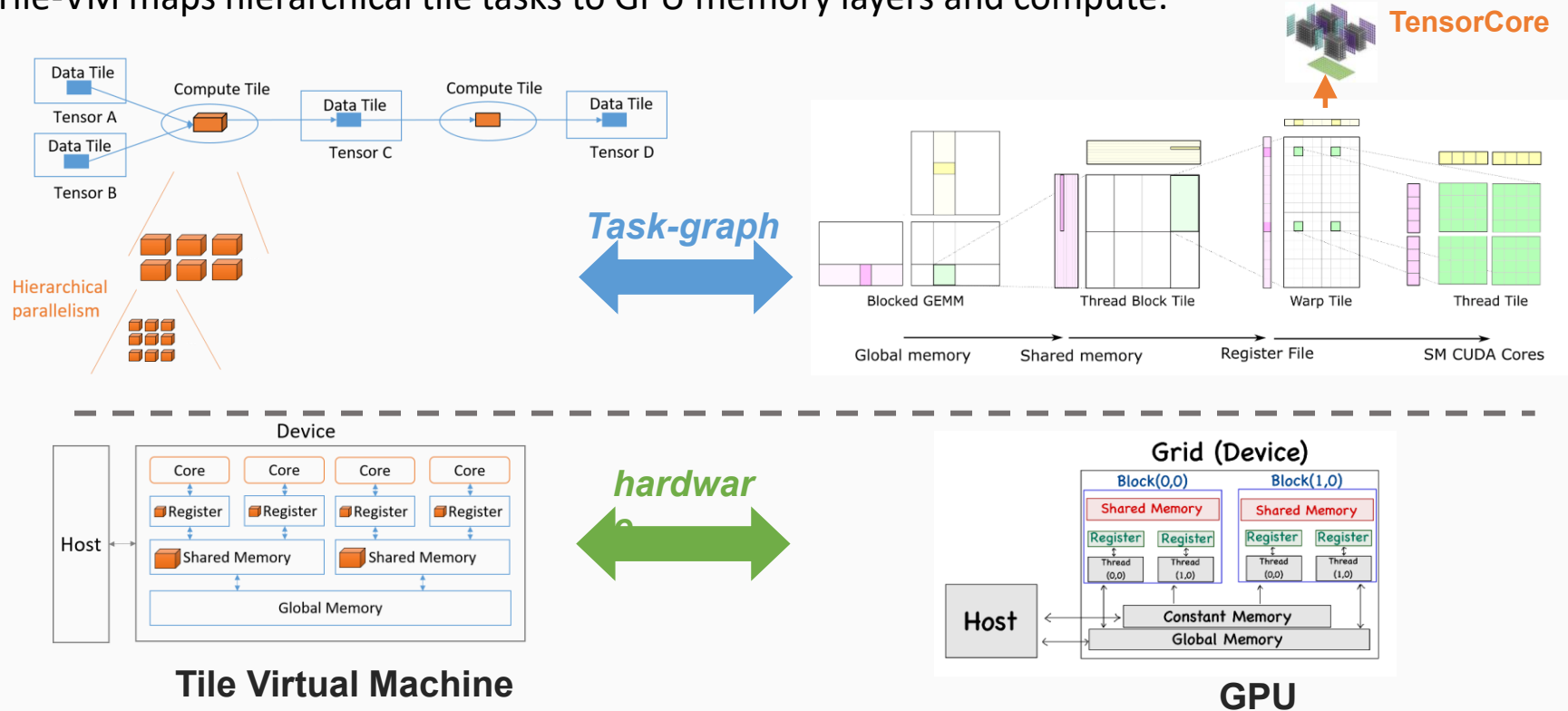


Tile Hardware Abstraction



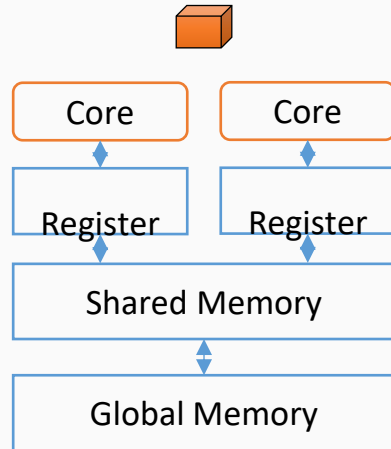
Example: Mapping Between Tile-VM and GPU

Tile-VM maps hierarchical tile tasks to GPU memory layers and compute.

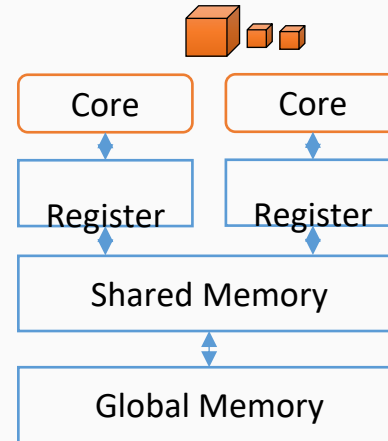


Tile-VM Control API: Tile Allocation

- `T.alloc_shared(L1)`

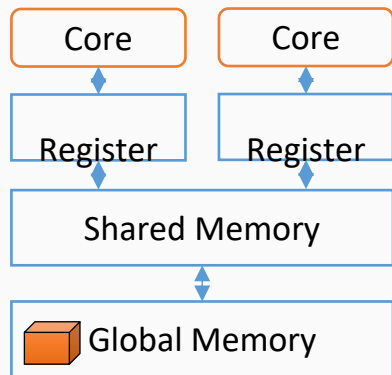


- `T.alloc_fragment(L0)`

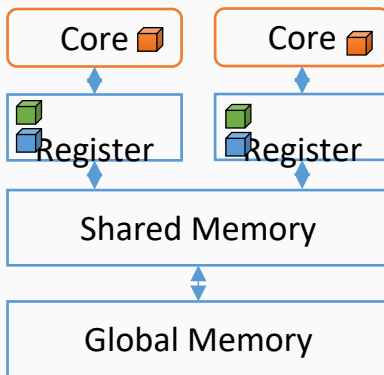


Tile-VM Control API: Tile Operators

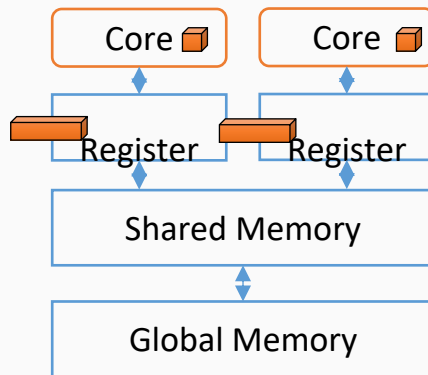
- `T.copy()`



- `T.gemm()`

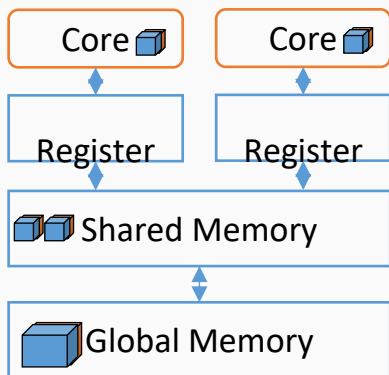


- `T.reduce()`

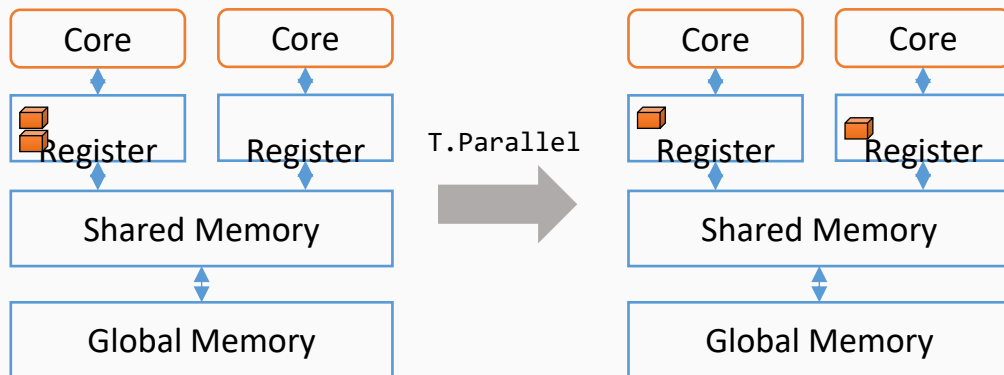


Tile-VM Control API: Tile Schedule

- `T.Pipelined()`



- `T.Parallel()`



TileLang Program: GEMM example

Triton Matmul Implementation

Triton

```
1 @triton.jit
2 def matmul_kernel(
3     a_ptr, b_ptr, c_ptr,
4     M, N, K,
5     stride_am, stride_ak,
6     stride_bk, stride_bn,
7     stride_cm, stride_cn,
8     BLOCK_SIZE_M: tl.constexpr, BLOCK_SIZE_N: tl.constexpr, BLOCK_SIZE_K: tl.constexpr,
9     GROUP_SIZE_M: tl.constexpr,
10    ACTIVATION: tl.constexpr
11 ):
12
13     pid = tl.program_id(axis=0)
14     num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
15     num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
16     num_pid_in_group = GROUP_SIZE_M * num_pid_n
17     group_id = pid // num_pid_in_group
18     first_pid_m = group_id * GROUP_SIZE_M
19     group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
20     pid_m = first_pid_m + (pid % num_pid_in_group) % group_size_m
21     pid_n = (pid % num_pid_in_group) // group_size_m
22
23     offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
24     offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
25     offs_k = tl.arange(0, BLOCK_SIZE_K)
26     a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak)
27     b_ptrs = b_ptr + (offs_k[:, None] * stride_bk + offs_bn[None, :] * stride_bn)
28
29     for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
30         a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k * BLOCK_SIZE_K, other=0.0)
31         b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k * BLOCK_SIZE_K, other=0.0)
32         accumulator = tl.dot(a, b, accumulator, out_dtype=tl.float16)
33         a_ptrs += BLOCK_SIZE_K * stride_ak
34         b_ptrs += BLOCK_SIZE_K * stride_bk
35
36     c = accumulator.to(tl.float16)
37
38     offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
39     offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
40     c_ptrs = c_ptr + stride_cm * offs_cm[:, None] + stride_cn * offs_cn[None, :]
41     c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
42     tl.store(c_ptrs, c, mask=c_mask)
43
```

TileLang

TileLang Matmul Implementation

```
1 @tilelang.jit
2 def Matmul(A: T.Buffer, B: T.Buffer, C: T.Buffer):
3     with T.Kernel(
4         ceildiv(N, block_N), ceildiv(M, block_M), threads=128
5     ) as (bx, by):
6         A_shared = T.alloc_shared((block_M, block_K))
7         B_shared = T.alloc_shared((block_K, block_N))
8         C_local = T.alloc_fragment((block_M, block_N), accum_dtype)
9
10        # Define Layout (Optional)
11        T.annotate_layout(
12            {
13                A_shared: your_customized_layout(A_shared),
14                B_shared: your_customized_layout(B_shared),
15            }
16        )
17        # Improve L2 Cache Locality (Optional)
18        T.use_swizzle(panel_size=10, order="row")
19
20        T.clear(C_local)
21        for k in T.Pipelined(ceildiv(K, block_K), num_stages=3):
22            T.copy(A[by * block_M, k * block_K], A_shared)
23            T.copy(B[k * block_K, bx * block_N], B_shared)
24            T.gemm(A_shared, B_shared, C_local)
25
26        T.copy(C_local, C[by * block_M, bx * block_N])
27
```

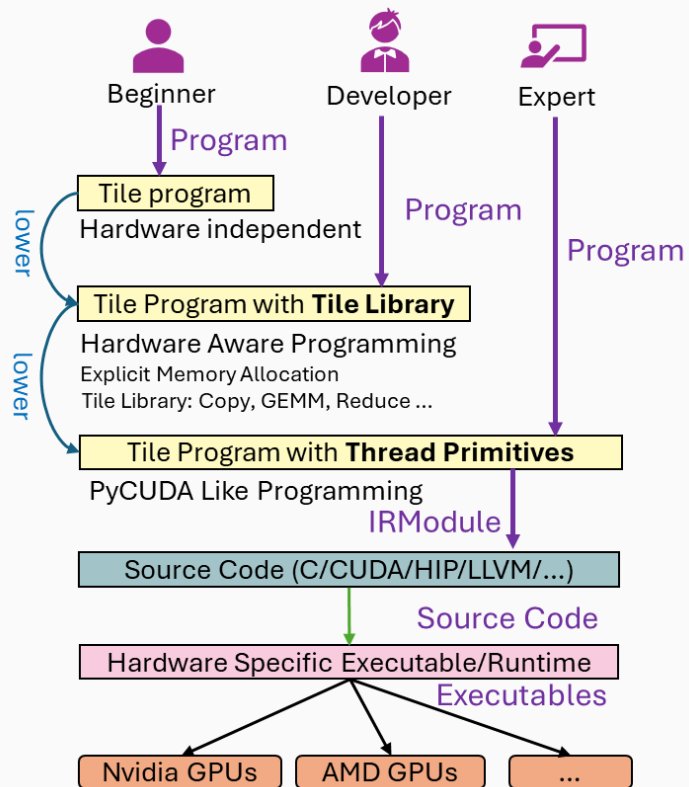
Define buffer on desired memory layer

Define desired layout (can be handled by TileLang by default)

L2 cache swizzle

Auto pipeline scheduling

TileLang Programming Modes



Beginner

$C[i, j] = A[i, k] + B[k, j]$

Simple Tensor Expression

Developer

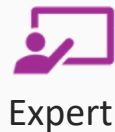
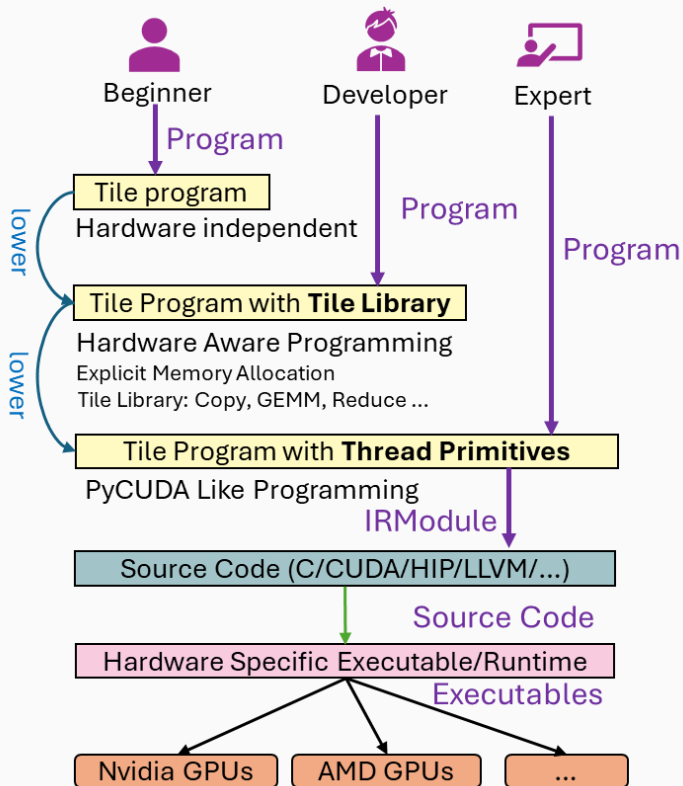
```
with T.Kernel(T.ceildiv(N, block_N), T.ceildiv(M, block_M), threads=128) as (bx, by):
    A_shared = T.alloc_shared((block_M, block_K), dtype)
    B_shared = T.alloc_shared((block_K, block_N), dtype)
    C_local = T.alloc_fragment((block_M, block_N), accum_dtype)

    T.clear(C_local)
    for k in T.Pipelined(T.ceildiv(K, block_K), num_stages=3):
        T.copy(A[by * block_M, k * block_K], A_shared)
        T.copy(B[k * block_K, bx * block_N], B_shared)
        T.gemm(A_shared, B_shared, C_local)

    T.copy(C_local, C[by * block_M, bx * block_N])
```

Block-level Tile Programming

TileLang Programming Modes



```
with T.Kernel(T.ceildiv(N, block_N), T.ceildiv(M, block_M), threads=threads) as (bx, by):
```

```
A_shared = T.alloc_shared(A_shared_shape, in_dtype, scope=shared_scope)
B_shared = T.alloc_shared(B_shared_shape, in_dtype, scope=shared_scope)
C_shared = T.alloc_shared(C_shared_shape, out_dtype, scope=shared_scope)
A_local = T.alloc_local((warp_rows * local_size_a), in_dtype)
B_local = T.alloc_local((warp_cols * local_size_b), in_dtype)
C_local = T.alloc_local((warp_rows * warp_cols * local_size_c), accum_dtype)
T.annotate_layout({
    A_shared: make_swizzle_layout(A_shared),
    B_shared: make_swizzle_layout(B_shared),
})

# Improve L2 Cache
T.use_swizzle(panel_size=10)
T.clear(C_local)
for ko in T.Pipelined((K // block_K), num_stages=stage):
    # Load A into shared memory
    for i, k in T.Parallel(block_M, block_K):
        A_shared[i, k] = A[by * block_M + i, ko * block_K + k]
    # Load B into shared memory
    for j, k in T.Parallel(block_N, block_K):
        B_shared[j, k] = B[bx * block_N + j, ko * block_K + k]
    for ki in T.serial(0, (block_K // micro_size_k)):
        # Load A into fragment
        mma_emitter.ldmatrix_a(A_local, A_shared, ki)
        # Load B into fragment
        mma_emitter.ldmatrix_b(B_local, B_shared, ki)
        # Perform Matrix Multiplication
        mma_emitter.mma(A_local, B_local, C_local)

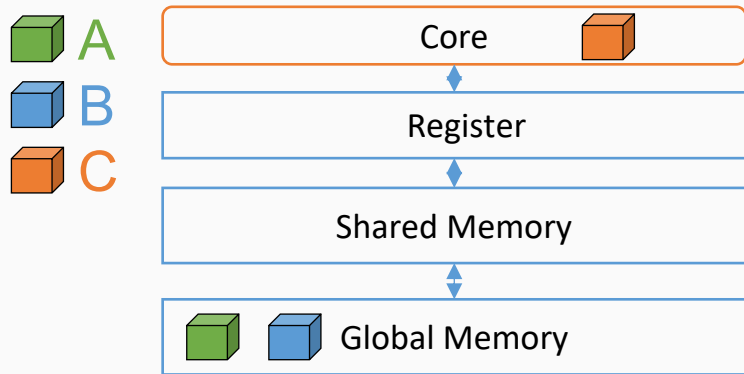
# Perform STMatrix
mma_emitter.stmatrix(C_local, C_shared)
# Store shared into global
for i, j in T.Parallel(block_M, block_N):
    C[by * block_M + i, bx * block_N + j] = C_shared[
        i // micro_size_x,
        j // micro_size_y,
        i % micro_size_x,
        j % micro_size_y,
    ]
]
```

Control thread
behavior via
thread-level
primitive



TileLang Example

TileLang Program: GEMM example



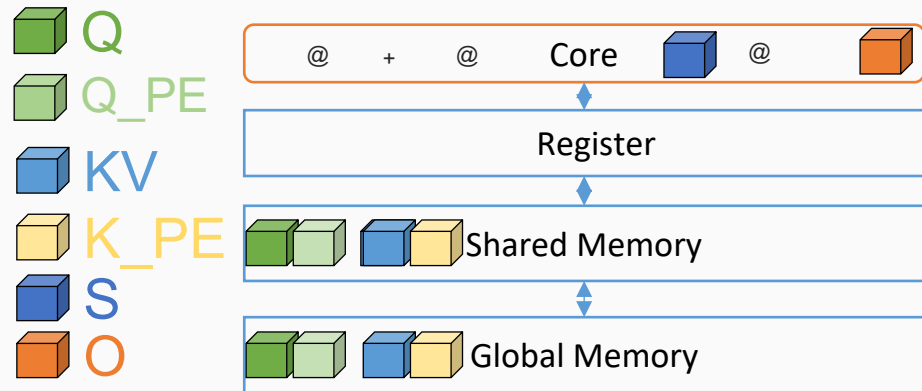
```
with T.Kernel(T.ceildiv(N, block_N), T.ceildiv(M, block_M), threads=128) as (bx, by):
    A_local = T.alloc_fragment((block_M, block_K), dtype)
    B_local = T.alloc_fragment((block_K, block_N), dtype)
    A_shared = T.alloc_shared((block_M, block_K), dtype)
    B_shared = T.alloc_shared((block_K, block_N), dtype)
    C_shared = T.alloc_shared((block_M, block_N), dtype)
    C_local = T.alloc_fragment((block_M, block_N), accum_dtype)

    T.clear(C_local)
    for k in T.Pipelined(T.ceildiv(K, block_K), num_stages=3):
        T.copy(A[by * block_M, k * block_K], A_shared)
        T.copy(B[k * block_K, bx * block_N], B_shared)
        T.copy(A_shared, A_local)
        T.copy(B_shared, B_local)

        T.gemm(A_local, B_local, C_local)

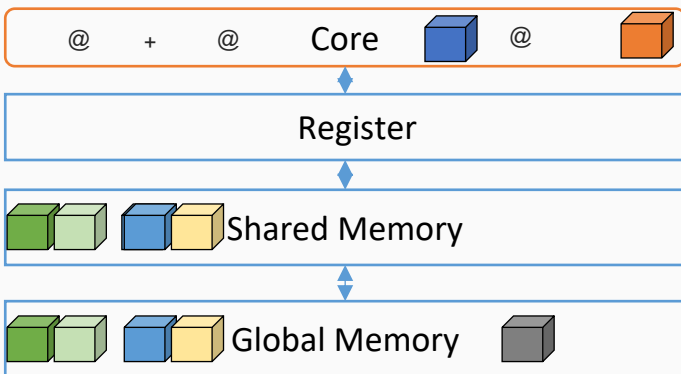
    T.copy(C_local, C_shared)
    T.copy(C_shared, C[by * block_M, bx * block_N])
```

TileLang Program: MLA example



```
T.copy(Q[bx, by * VALID_BLOCK_H:(by + 1) * VALID_BLOCK_H, :], Q_shared)
T.copy(Q_pe[bx, by * VALID_BLOCK_H:(by + 1) * VALID_BLOCK_H, :], Q_pe_shared)
for k in T.Pipelined(loop_range, num_stages=2):
    T.copy(KV[bx, k * block_N:(k + 1) * block_N, cur_kv_head, :], KV_shared)
    T.copy(K_pe[bx, k * block_N:(k + 1) * block_N, cur_kv_head, :], K_pe_shared)
    T.clear(acc_s)
    f.gemm(
        Q_shared, KV_shared, acc_s, transpose_B=True, policy=T.GemmWarpPolicy.FullCol)
    T.gemm(
        Q_pe_shared,
        K_pe_shared,
        acc_s,
        transpose_B=True,
        policy=T.GemmWarpPolicy.FullCol)
    ...
    T.copy(acc_s, S_shared)
    ...
    T.gemm(S_shared, KV_shared, acc_o, policy=T.GemmWarpPolicy.FullCol)
T.copy(acc_o, O_shared)
T.copy(O_shared, Output[bx, by * VALID_BLOCK_H:(by + 1) * VALID_BLOCK_H, :])
```

TileLang Program: Sparse MLA example



```
T.copy(Q[b_i, s_i, H0:H1, :D], Q_shared)
T.copy(Q[b_i, s_i, H0:H1, D:], Q_tail_shared)
```

```
for i_i in T.Pipelined(NI, num_stages=num_stages):
    for bi_i in T.Parallel(BI):
        mask[bi_i] = Indices[b_i, s_i, g_i, i_i * BI + bi_i] <= max_kv_i
    for bi_i, d_i in T.Parallel(BI, D):
        KV_shared[bi_i, d_i] = KV[b_i, Indices[b_i, s_i, g_i, i_i * BI + bi_i], g_i, d_i]
    for bi_i, d_i in T.Parallel(BI, D_tail):
        K_tail_shared[bi_i, d_i] = KV[b_i, Indices[b_i, s_i, g_i, i_i * BI + bi_i], g_i, D + d_i]
    for h_i, bi_i in T.Parallel(H_per_block, BI):
        acc_s[h_i, bi_i] = T.if_then_else(mask[bi_i], 0, -T.infinity(acc_s.dtype))

    .gemm(
        Q_shared,
        KV_shared,
        acc_s,
        transpose_B=True,
        policy=T.GemmWarpPolicy.FullRow,
    )
    T.gemm(
        Q_tail_shared,
        K_tail_shared,
        acc_s,
        transpose_B=True,
        policy=T.GemmWarpPolicy.FullRow,
    )
    ...
    T.copy(acc_s, S_shared)
    T.gemm(S_shared, KV_shared, acc_o, policy=T.GemmWarpPolicy.FullRow)
    T.copy(acc_o, O_shared)
    T.copy(acc_o, Output[b_i, s_i, H0:H1, :])
```

Learning TileLang with 10 puzzles!

<https://github.com/tile-ai/tilelang-puzzles>



TileLang Puzzles

TileLang Puzzles is a set of puzzles to help you learn [TileLang](#), a domain-specific language for developing high-performance deep learning kernels. We will start from some trivial examples and smoothly progress to modern kernels such as GEMM and FlashAttention, aiming to provide a comprehensive understanding of the design principles of TileLang.

Getting Started

Environment Configuration

The only thing you need to install is [TileLang](#) and its dependency. To check your installation, run:

```
python -c "import tilelang; print(tilelang.__version__);"
```

You can also run our environment check script to confirm that TileLang and your GPU are configured correctly:

```
python3 scripts/check_tilelang_env.py
```

Run Puzzles

We provide 10 puzzles in the `puzzles/` directory, each as a standalone executable script. Reference implementations are available in `ans/` for comparison. To run a puzzle, use:

[tilelang-puzzles](#) / [puzzles](#) /

3 people [Release] Initialize version (#2)

Name	Last commit message
..	
01-copy.py	[Release] Initialize version (#2)
02-vector-add.py	[Release] Initialize version (#2)
03-outer-vec-add.py	[Release] Initialize version (#2)
04-backward-op.py	[Release] Initialize version (#2)
05-reduce-sum.py	[Release] Initialize version (#2)
06-softmax.py	[Release] Initialize version (#2)
07-scalar-flash-attn.py	[Release] Initialize version (#2)
08-matrix.py	[Release] Initialize version (#2)
09-conv.py	[Release] Initialize version (#2)
10-dequant-mm.py	[Release] Initialize version (#2)

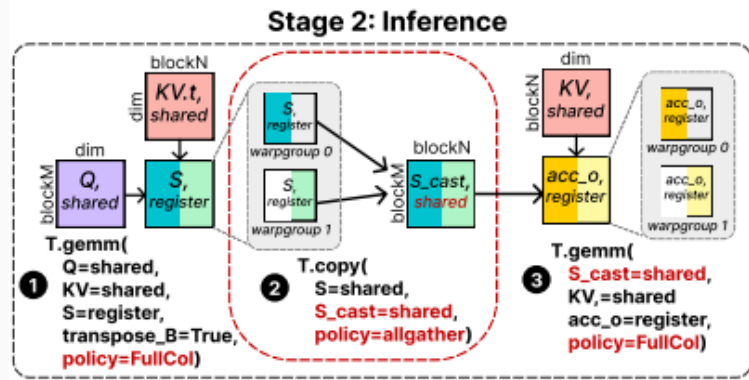
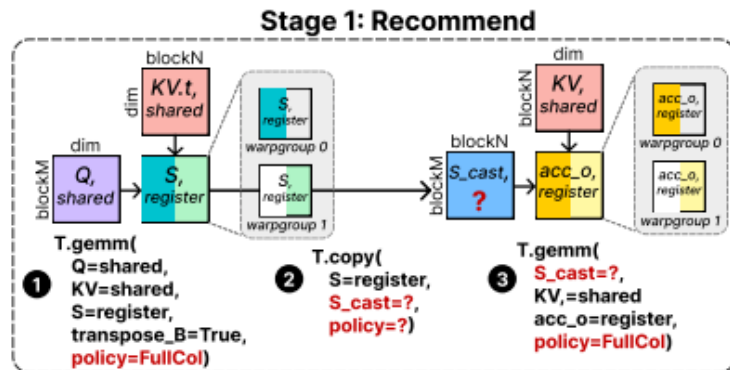


TileLang Mechanism

```

1 ❶ Decide tile configuration Recommend
2 with T.Kernel(batch, heads // min(block_H, kv_group_num), threads=256) as (bx, by):
3 ❷ Define buffer on desired memory layer Recommend
4 Q_shared = T.alloc_shared([block_H, dim], dtype)
5 S_shared = T.alloc_shared([block_H, block_N], dtype)
6 Q_pe_shared = T.alloc_shared([block_H, pe_dim], dtype)
7 acc_s = T.alloc_fragment([block_H, block_N], accum_dtype)
8 # skip initialization statements
9 ❸ Define desired data layout Inference
10 T.annotate_layout({{0_shared: T.make_swizzled_layout(0_shared)}})
11
12 T.copy(Q[bx, by * block_H:(by + 1) * block_H, :], Q_shared)
13 T.copy(Q_pe[bx, by * block_H:(by + 1) * block_H, :], Q_pe_shared)
14
15 ❹ Auto pipeline scheduling Inference
16 for k in T.Pipeline(T.ceildiv(seqlen_kv, block_N), num_stages=2):
17     T.copy(KV[bx, k * block_N:(k + 1) * block_N, cur_kv_head, :], KV_shared)
18     T.copy(K_pe[bx, k * block_N:(k + 1) * block_N, cur_kv_head, :], K_pe_shared)
19     T.clear(acc_s)
20
21 ❺ Warp partitioning Recommend
22 T.gemm(Q_shared, KV_shared, acc_s, transpose_B=True,
23        policy=T.GemmWarpPolicy.FullCol)
24 T.gemm(Q_pe_shared, K_pe_shared, acc_s, transpose_B=True,
25        policy=T.GemmWarpPolicy.FullCol)
26 # skip flash operations on scaling and softmax
27 T.copy(acc_s, S_shared)
28
29 ❻ Auto utilize high performance instruction Inference
30 for i, j in T.Parallel(block_H, dim):
31     acc_o[i, j] += logsum[i]
32 T.copy(acc_o, 0_shared)
33 T.copy(0_shared, Output[bx, by * block_H:(by + 1) * block_H, :])
34

```

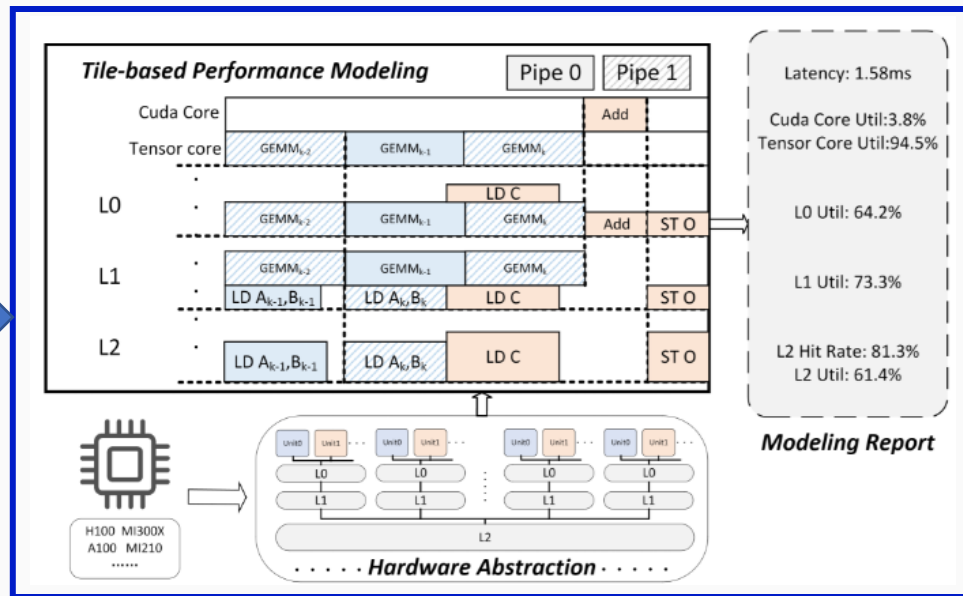


TileLang Core Mechanisms—Tile Evaluation

Introduces a cost model to evaluate tile configurations, enabling to identify high-performance scheduling strategies.

```
T.copy(Q[bx, by * VALID_BLOCK_H:(by + 1) * VALID_BLOCK_H, :], Q_shared)
T.copy(Q_pe[bx, by * VALID_BLOCK_H:(by + 1) * VALID_BLOCK_H, :], Q_pe_shared)
for k in T.Pipelined(loop_range, num_stages=2):
    T.copy(KV[bx, k * block_N:(k + 1) * block_N, cur_kv_head, :], KV_shared)
    T.copy(K_pe[bx, k * block_N:(k + 1) * block_N, cur_kv_head, :], K_pe_shared)
    T.clear(acc_s)
    T.gemm(
        Q_shared, KV_shared, acc_s, transpose_B=True, policy=T.GemmWarpPolicy.FullCol)
    T.gemm(
        Q_pe_shared,
        K_pe_shared,
        acc_s,
        transpose_B=True,
        policy=T.GemmWarpPolicy.FullCol)
    ...
    T.copy(acc_s, S_shared)
    ...
    T.gemm(S_shared, KV_shared, acc_o, policy=T.GemmWarpPolicy.FullCol)
T.copy(acc_o, O_shared)
T.copy(O_shared, Output[bx, by * VALID_BLOCK_H:(by + 1) * VALID_BLOCK_H, :])
```

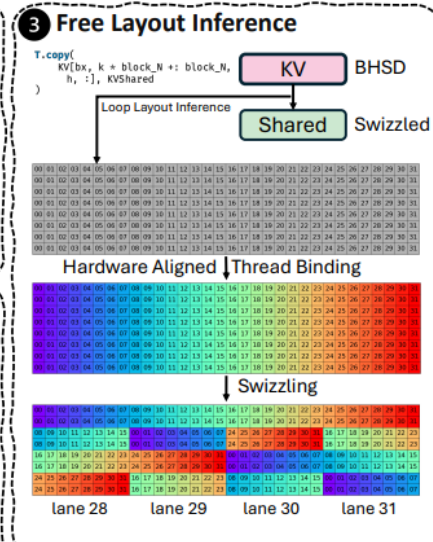
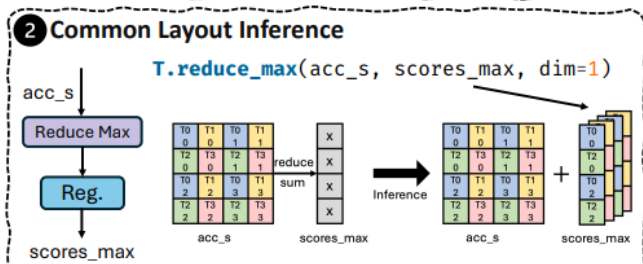
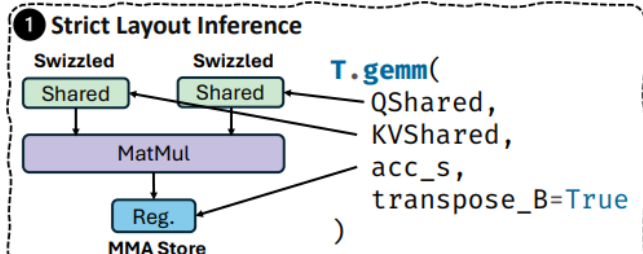
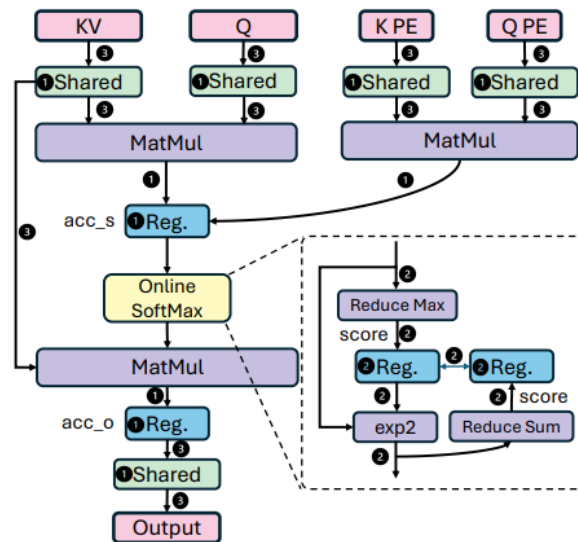
Tile Program



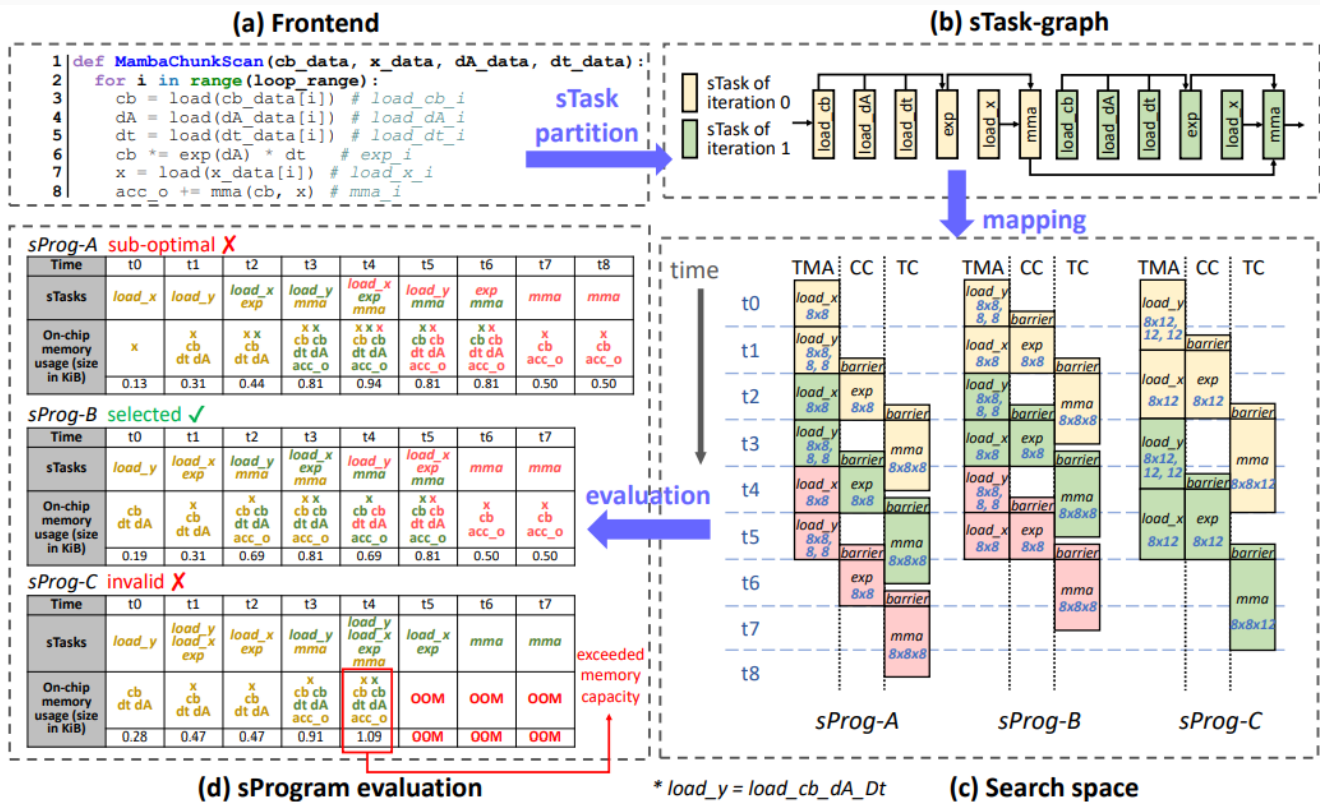
Tile Cost Model

TileLang Core Mechanisms—Tile Inference

The system further incorporates **automated layout inference**—including tiling decisions—and advanced techniques for memory placement and pipeline control, significantly improving kernel efficiency with minimal manual effort.



TileLang Core Mechanisms—Tile Pipeline





TileLang Impl&Eval

```

1 @T.prim_func
2 def Matmul(
3     A: T.Tensor((M, K), "float16"),
4     B: T.Tensor((K, N), "float16"),
5     C: T.Tensor((M, N), "float16"),
6 ):
7     with T.Kernel(
8         ceildiv(M, block_M),
9         ceildiv(M, block_M), threads=threads) as (bx, by):
10
11         # Buffer Allocation
12         A_shared = T.alloc_shared((block_M, block_K), "float16")
13
14         B_shared = T.alloc_shared((block_K, block_N), "float16")
15
16         C_local = T.alloc_fragment((block_M, block_N), "float16")
17
18         # Initialize C_local
19         T.clear(C_local)
20
21         # Main Loop with Expanded Pipeline
22         T.copy(A[by * block_M, 0 * block_K], A_shared)
23
24         T.copy(B[0 * block_K, bx * block_N], B_shared)
25
26         # Main Loop with Pipeline Annotation
27         for k in T.serial(ceildiv(K, block_K)):
28             T.gemm(A_shared, B_shared, C_local)
29
30             T.copy(A[by * block_M, k * block_K], A_shared)
31
32             T.copy(B[k * block_K, bx * block_N], B_shared)
33
34         # Compute the last stage
35         T.gemm(A_shared, B_shared, C_local)
36         # Copy the result to the output buffer
37         T.copy(C_local, C[by * block_M, bx * block_N])

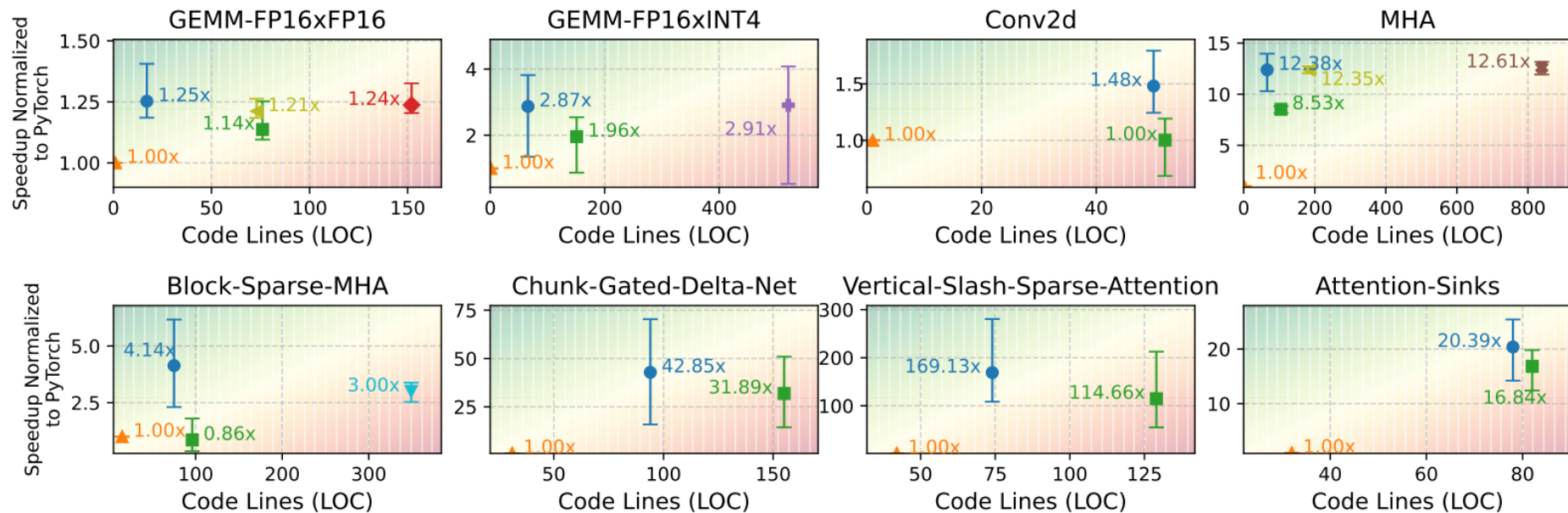
```

```

1 @T.prim_func
2 def Matmul(
3     A: T.Tensor((M, K), "float16"), B: T.Tensor((K, N), "float16"),
4     C: T.Tensor((M, N), "float16"),
5 ):
6     A_shared = T.decl_buffer((block_M * block_K), dtype, scope="shared")
7     B_shared = T.decl_buffer((block_K * block_N), dtype, scope="shared")
8     C_local = T.decl_buffer((128,), accum_dtype, scope="local")
9     bx = T.thread_binding(T.ceildiv(M, block_M), "blockIdx.x")
10    by = T.thread_binding(T.ceildiv(M, block_M), "blockIdx.y")
11    tid = T.thread_binding(threads, "threadIdx.x")
12    for i in T.unroll(128):
13        C_local[i] = T.float32(0)
14    for i in T.unroll(4):
15        T.ptx_cp_async("uint8", A_shared.data, 0, A.data, 0, 16)
16        T.ptx_cp_async("uint8", B_shared.data, 0, B.data, 0, 16)
17    T.ptx_commit_group()
18    for ko in T.serial(ceildiv(K, block_K) - 1):
19        T.ptx_wait_group(0)
20        T.call_extern(
21            "tl::gemv_ss<128, 128, 32, 2, 2, 0, 0>",
22            T.tvn_access_ptr(A_shared.data, 0, block_M * block_K, 1),
23            T.tvn_access_ptr(B_shared.data, 0, block_K * block_N, 1),
24            T.tvn_access_ptr(C_local.data, 0, 128, 3),
25        )
26    for i in T.unroll(4):
27        T.ptx_cp_async("uint8", A_shared.data, 0, A.data, 0, 16)
28        T.ptx_cp_async("uint8", B_shared.data, 0, B.data, 0, 16)
29    T.ptx_commit_group()
30    T.ptx_wait_group(0)
31    T.call_extern(
32        "tl::gemv_ss<128, 128, 32, 2, 2, 0, 0>",
33        T.tvn_access_ptr(A_shared.data, 0, block_M * block_K, 1),
34        T.tvn_access_ptr(B_shared.data, 0, block_K * block_N, 1),
35        T.tvn_access_ptr(C_local.data, 0, 128, 3),
36    )
37    ...

```

Performance and Code Complexity on NVIDIA H100 GPU





What's Next: TileLang->Tile AI



tile-ai

Enabling Lightning-Fast AI Workloads Development via Tiling

360 followers

tile-ai@outlook.com

Overview Repositories 17 Projects 2 Packages People 1

应用层：大模型、智能体、AI4Science等

TileOps
大模型算子库

TileRT
面向**极限速度**的推理引擎

TileScale
面向**分布式**架构的编程和编译器

TileLang
面向**大模型和新硬件架构**的编程和编译器

TileSight
大模型性能分析模型

硬件：最新GPU架构、近存计算、存内计算、晶圆级芯片等

tilelang-puzzles (Public)

Learning TileLang with 10 puzzles!

Python 18 1 0 0 Updated 1 hour ago

tilelang-ascend (Public)

Ascend TileLang adapter

C++ 200 71 51 22 Updated 1 hour ago

tilelang (Public)

Domain-specific language designed to streamline the development of high-performance GPU/CPU/Accelerators kernels

Python 4,845 423 65 (5 issues need help) 9 Updated 1 hour ago

whl (Public)

HTML 0 0 0 Updated 15 hours ago

tilelang-nightly (Public)

Python 6 6 0 0 Updated 15 hours ago

tvm (Public)

Forked from apache/tvm

Open deep learning compiler stack for cpu, gpu and specialized accelerators

Python 19 3,835 0 1 Updated yesterday

TileOps (Public)

Python 80 21 36 7 Updated yesterday

BitBLAS (Public)

ReDesign of <https://github.com/microsoft/BitBLAS>

Python 8 1 1 0 Updated 2 days ago

tilescale (Public)

Forked from tile-ai/tilelang

Tile-based language built for AI computation across all scales

C++ 118 422 8 5 Updated 2 days ago

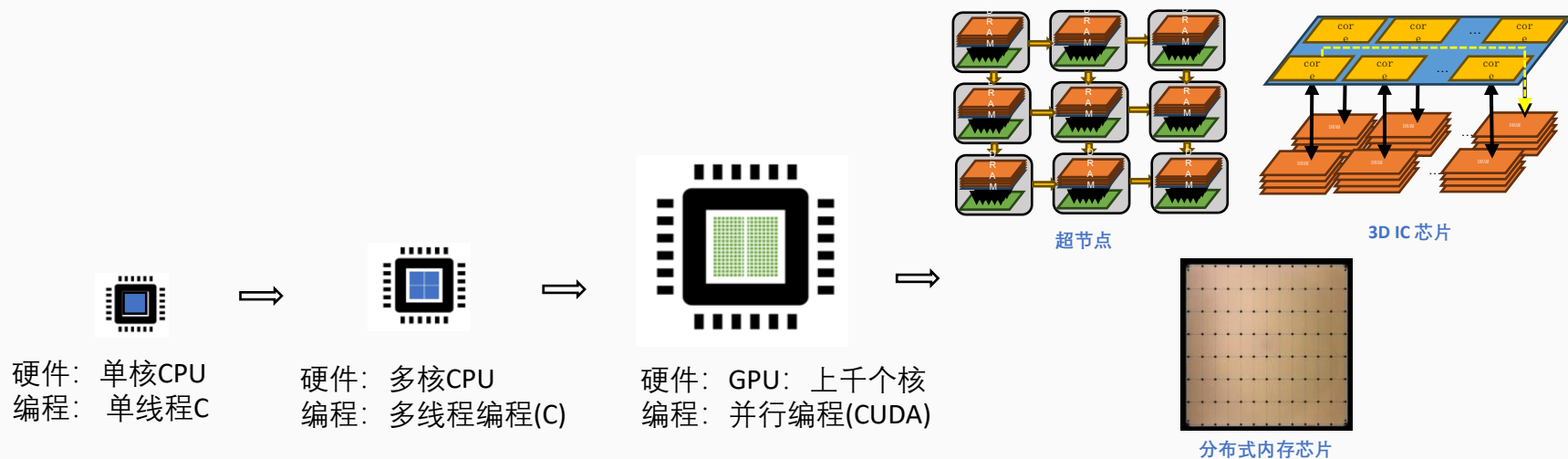
TileRT (Public)

Tile-Based Runtime for Ultra-Low-Latency LLM Inference



TileScale: TileLang for AI computation across all scales

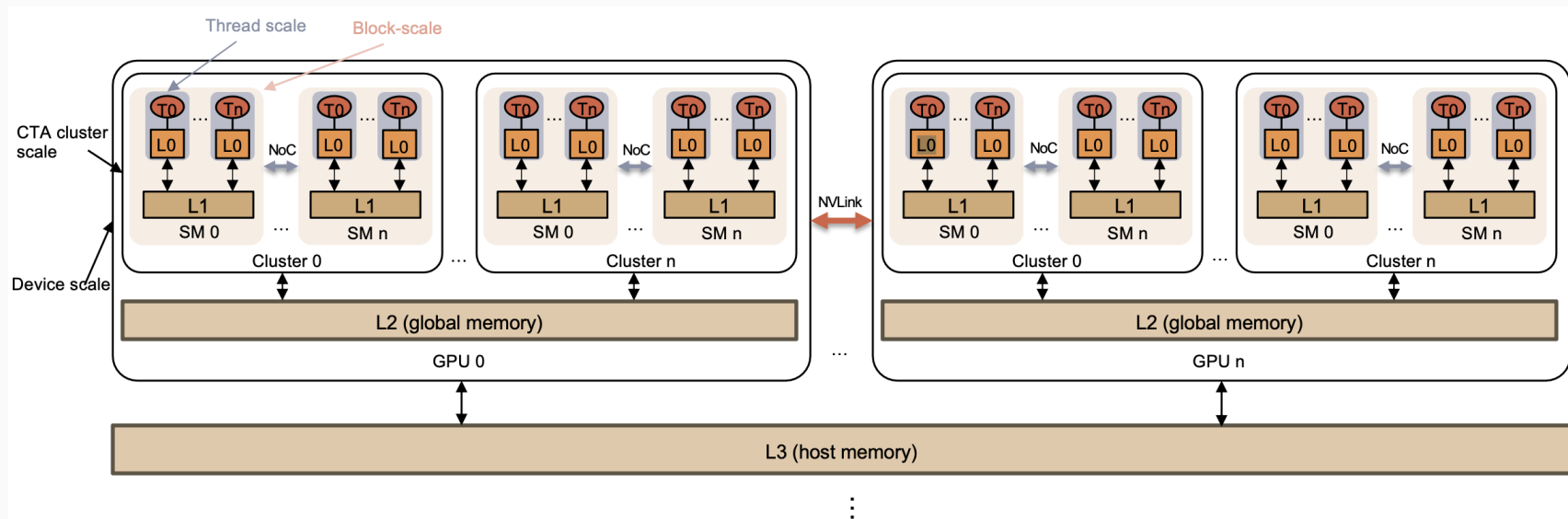
- 当前芯片制程发展放缓，AI计算系统正在全面走向分布式化：多GPU、超级点、分布式内存等
- 过去每十年计算架构和软件栈都会发一次迁移性变化
- 从大模型通算融合场景出发，结合新架构的需求，探索面向分布式架构的编程软件栈



硬件：多芯片超节点/3D IC
编程：分布式编程？

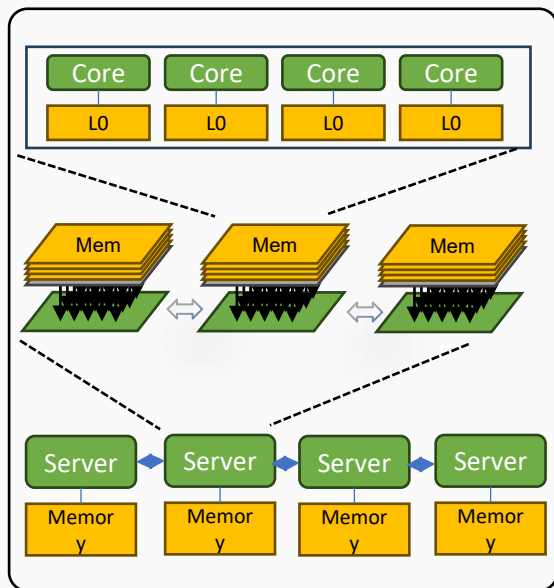
卡内和卡间的统一分布式硬件抽象

目标：把卡内和卡间层的互联网络、计算单元、分布式内存虚拟化成一个统一的“大GPU”



TileScale: 面向大模型的分布式编程和编译器

在统一的“大GPU”上提供Tile级的编程接口



用户通过高级语言编写通信与计算逻辑

```
with T.Scale("device"):  
    A_global = T.view(A, layout=T.FullCol)  
    B_global = T.view(B, layout=T.FullRow)  
    C_global = T.view(C, layout=T.Replica)  
  
with T.Scale("block"):  
    A_local = T.alloc((block_M, block_K), dtype, level="l0")  
    B_local = T.alloc((block_K, block_N), dtype, level="l0")  
    C_local = T.alloc((block_M, block_N), accum_dtype, level="l0")  
    T.clear(C_local)  
  
for k in T.Pipelined(T.ceildiv(A_global.shape[1], block_K), num=  
    with T.Scale("warpgroup") as wg_id, wg_num:  
        A_local_wg = T.view(A_local, layout=T.FullCol)  
        B_local_wg = T.view(B_local, layout=T.FullRow)  
        C_local_wg = T.view(C_local, layout=T.Replica)  
        T.copy(A_local_wg, A_global[by * block_M, k * block_K])  
        T.copy(B_local_wg, B_global[k * block_K, bx * block_N])  
        T.gemm(A_local_wg, B_local_wg, C_local_wg)
```

编译器自动生成底层执行代码

```
// Load q to shared memory  
#pragma unroll  
for (int i_q = 0; i_q < 8; ++i_q) {  
    int smem_q_offset =  
        (((tidx & 63) >> 3) * 2048)  
        + (i_q * 128)  
        + ((tidx >> 8) * 64)  
        + (((tidx & 7) >> 2) + ((i_q & 3) >> 1) & 1) * 32)  
        + (((tidx & 3) >> 1) + (i_q & 1) & 1) * 16)  
        + (((tidx >> 6) + (tidx & 1) & 1) * 8);  
  
    int q_offset = (i_q * 1024) + (tidx * 8);  
  
    tl::cp_async_gs<16>(  
        shared_storage.flash1a_utmmlcta.smem_q + smem_q_offset,  
        params.ptr_q + q_offset  
    );  
}  
  
// Load q positional encoding to shared memory  
int smem_q_pe_offset =  
    ((tidx >> 3) * 64)  
    + (((tidx & 63) >> 5) + ((tidx & 7) >> 2) & 1) * 32)  
    + (((tidx & 31) >> 4) + ((tidx & 3) >> 1) & 1) * 16)  
    + (((tidx & 15) >> 3) + (tidx & 1) & 1) * 8);  
  
int q_pe_offset = tidx * 8;  
  
tl::cp_async_gs<16>(  
    shared_storage.flash1a_utmmlcta.smem_q_pe + smem_q_pe_offset,  
    params.ptr_q_pe + q_pe_offset  
);
```

Tile-based AI Compute at All Scales: <https://github.com/tile-ai/tilescale>

Code Example

- Remote write

```
@T.prim_func
def main(
    dst: T.Tensor((M), "float32"),
    src: T.Tensor((M), "float32"),
):
    with T.Kernel(T.ceildiv(M, block_M), threads=threads) as (bx):
        rank = T.alloc_local([1], "uint64")
        num_rank = T.alloc_local([1], "uint64")
        rank[0] = T.get_rank()
        num_rank[0] = T.get_num_ranks()
        T.put_block(
            src=T.address_of(src[bx * block_M]),
            dst=T.address_of(dst[bx * block_M]),
            size=block_M,
            dst_pe=rank[0] ^ 1,
        )

return main
```

- Allgather-GEMM

```
T.copy(A[by * block_M, bx * block_K], A_shared)
T.copy(A_shared, A_ag[mype[0] * M, bx * block_K])
for k in T.serial(PE_num - 1):
    peer[0] = (mype[0] + 1 + k) % npes[0]
    T.putmem_signal_nbi_block(
        T.address_of(A_ag[mype[0] * M, 0]), T.address_of(A[0, 0]),
        block_M * block_K * 2, T.address_of(signal[k]), k + 1, 9, peer[0])
for k in T.serial(PE_num - 1):
    T.signal_wait_until(T.address_of(signal[k]), 0, k + 1)

for bk in T.serial(PE_num):
    T.clear(C_local)
    for k in T.Pipelined(T.ceildiv(K, block_K), num_stages=0):
        T.copy(A_ag[bk * M, k * block_K], A_shared)
        T.copy(B[k * block_K, bx * block_N], B_shared)
        T.gemm(A_shared, B_shared, C_local)
    T.copy(C_local, C[bk * M, bx * block_N])
```

TileScale Performance

- EP-dispatch

	TileScale	DeepEP
Time (ms)	1.1339	1.0003

- EP-combine

	TileScale	DeepEP
Time (ms)	1.0778	1.1504

- Allgather-GEMM

	TileScale	Triton-dist
Time (ms)	0.69	0.77

- GEMM-ReduceScatter

	TileScale	Triton-dist	Torch
Time (ms)	0.92	0.92	1.09

- sp_ag_attention

	TileScale	Triton-dist	Torch
Time (ms)	1.17	1.28	5.11

- Dequant GEMM

	TileScale	CUTLASS
Time (ms)	2.63	3.52



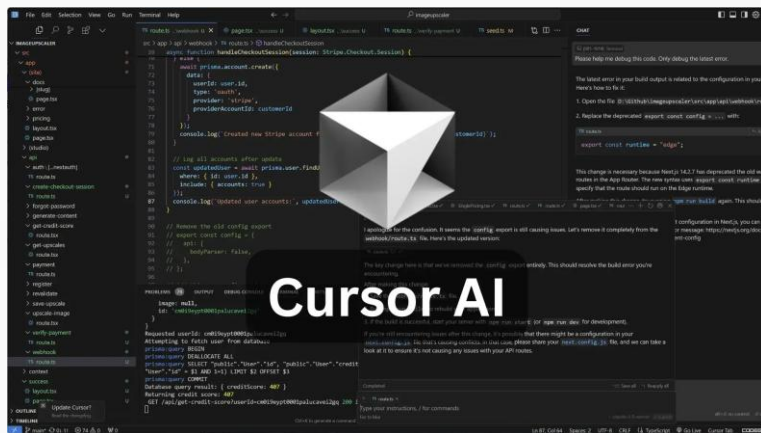
TileRT: Tile-Based Runtime for Ultra-Low-Latency LLM Inference



有什么可以帮忙的？

+ 询问任何问题

公司知识库

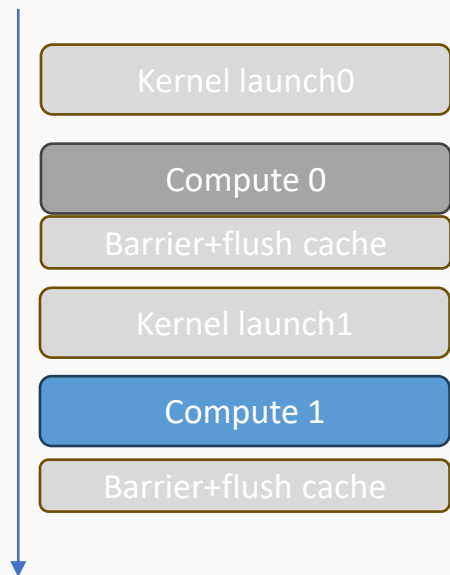


Coding agent, Enterprise AI Automation, 智能客服等

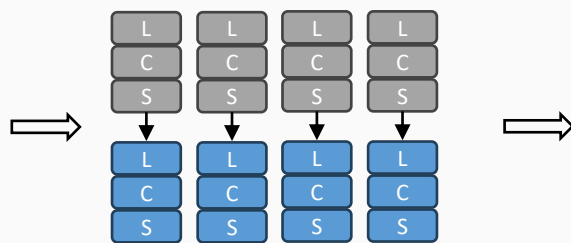
大量高价值场景，相比token成本，任务完成**质量**和**速度**越来越重要！

TileRT: 低延迟大模型推理引擎

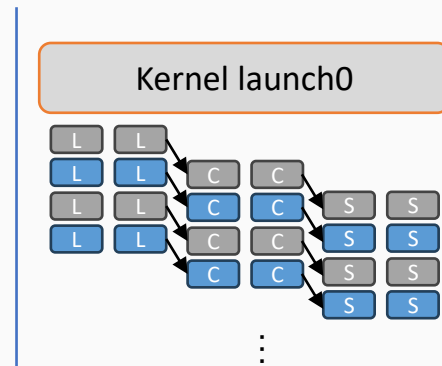
随着计算速度和内存带宽的提升，kernel启动及同步时间会远大于计算时间，模型的速度无法继续提升
例如：H100 kernel启动时间在3us左右，同步在1-2us左右；大模型的算子平均时间已到2-5us
方案：打破kernel的概念，直接调度tile级任务



现有GPU运行方式



算子分解为Tile级任务



启动一个引擎kernel，调度异构的Tile级任务（计算、IO、通信、barrier）

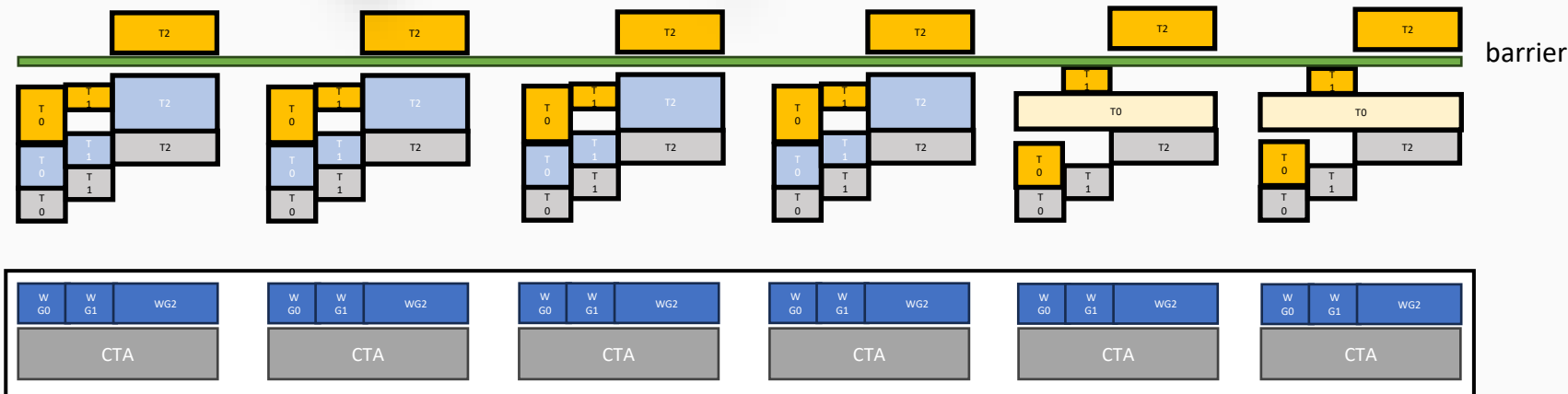
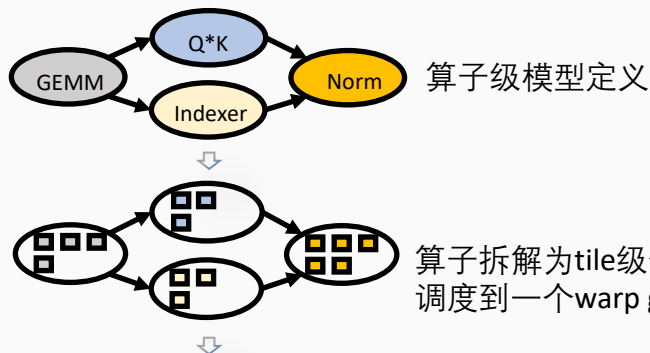
TileRT引擎运行方式

TileRT引擎设计

Tile级任务调度:

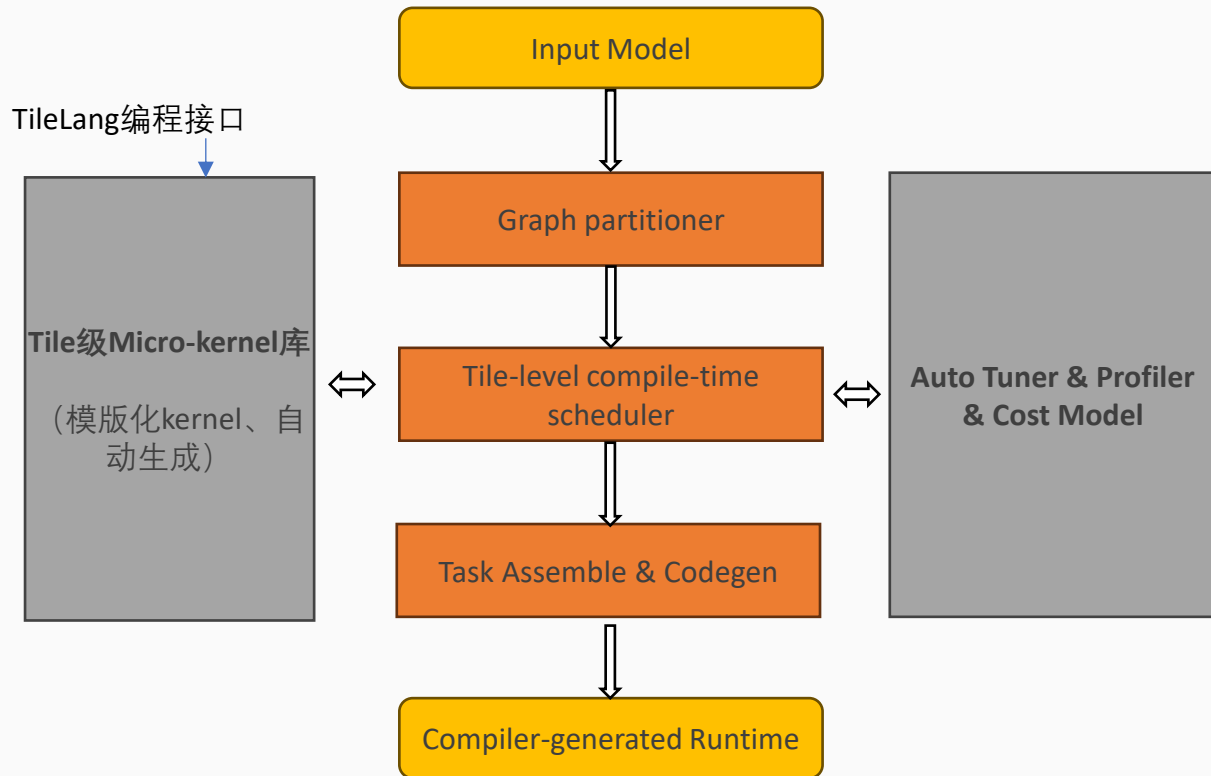
需要非常精细化的排布和
tuning

执行时间



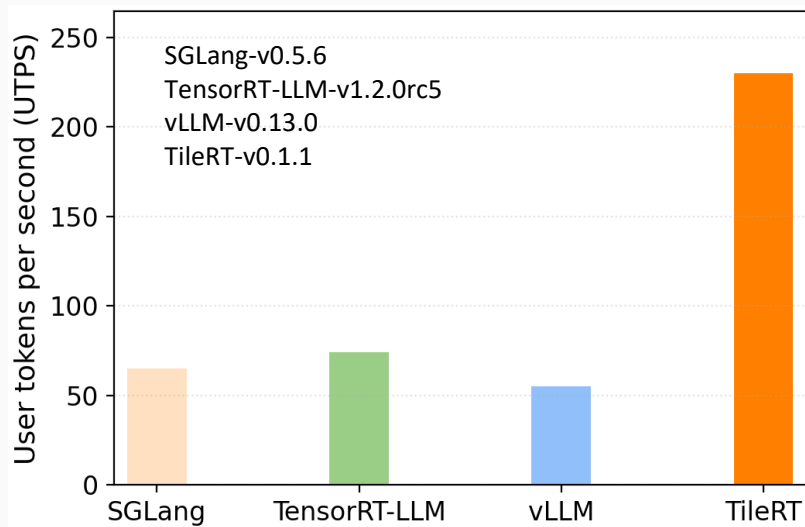
异构Worker虚拟化 (warp/block specialization)

TileRT框架: 编译器生成“运行引擎”



早期性能

项目仓库: <https://github.com/tile-ai/TileRT>



DeepSeek V3.2 TPOT速度对比(w/o MTP)



Chat with DeepSeek-V3.2-Exp

Message

Hello, please provide an overview of the history of artificial intelligence's great development.



Send

Real-time Speed



Chat with DeepSeek-V3.2-Exp

Message

Hello, please write two sort programs in C programming language. Thank you.



Send

Real-time Speed

(w/o MTP)

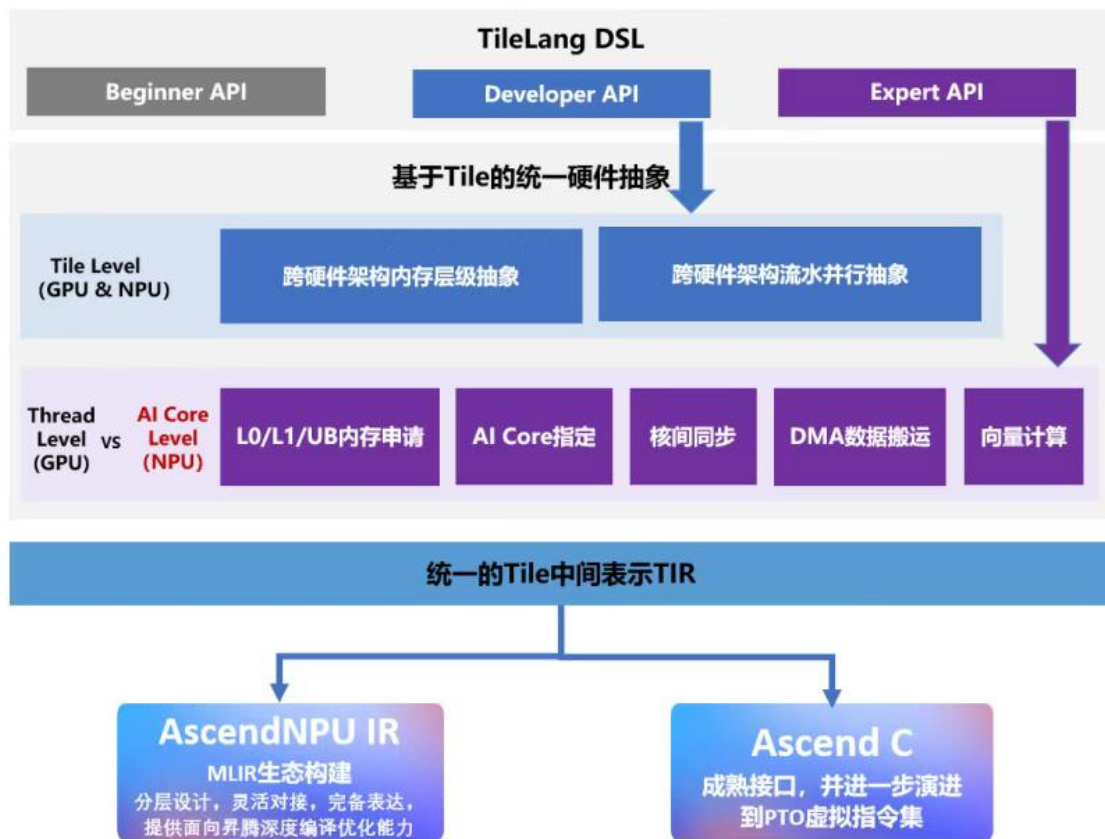
(w MTP)

DeepSeek V3.2推理演示 (TileRT)



TileLang-ascend: Ascend TileLang adapter

<https://github.com/tile-ai/tilelang-ascend>



GPU

```
1 @tilelang.jit(out_idx=[-1])
2 def matmul(M, N, K, block_M, block_N, block_K, dtype=T.float16, accum_dtype='float16'):
3     @T.prim_func
4     def gemm(
5         A: T.Tensor((M, K), dtype),
6         B: T.Tensor((K, N), dtype),
7         C: T.Tensor((M, N), dtype),
8     ):
9         with T.Kernel(T.ceildiv(N, block_N), T.ceildiv(M, block_M), threads=128):
10             A_shared = T.alloc_shared((block_M, block_K), dtype)
11             B_shared = T.alloc_shared((block_K, block_N), dtype)
12             C_local = T.alloc_fragment((block_M, block_N), accum_dtype)
13             T.clear(C_local)
14             for k in T.Pipelined(T.ceildiv(K, block_K), num_stages=3):
15                 T.copy(A[by * block_M, k * block_K], A_shared)
16                 T.copy(B[k * block_K, bx * block_N], B_shared)
17                 T.gemm(A_shared, B_shared, C_local)
18                 T.copy(C_local, C[by * block_M, bx * block_N])
19     return gemm
```

NPU

```
1 @tilelang.jit(out_idx=[-1], target="npuir")
2 def matmul(M, N, K, block_M, block_N, block_K, dtype="float16", accum_dtype='float16'):
3     @T.prim_func
4     def gemm(
5         A: T.Tensor((M, K), dtype),
6         B: T.Tensor((K, N), dtype),
7         C: T.Tensor((M, N), dtype),
8     ):
9         with T.Kernel(T.ceildiv(N, block_N) * T.ceildiv(M, block_M), is_npu=True):
10             by = cid // T.ceildiv(N, block_N)
11             bx = cid % T.ceildiv(N, block_N)
12             A_shared = T.alloc_shared((block_M, block_K), dtype)
13             B_shared = T.alloc_shared((block_K, block_N), dtype)
14             C_local = T.alloc_fragment((block_M, block_N), accum_dtype)
15             for k in T.Pipelined(T.ceildiv(K, block_K), num_stages=2):
16                 T.copy(A[by * block_M, k * block_K], A_shared)
17                 T.copy(B[k * block_K, bx * block_N], B_shared)
18                 T.gemm(A_shared, B_shared, C_local, initC=(k == 0))
19                 T.copy(C_local, C[by * block_M, bx * block_N])
20     return gemm
```

THANKS